Notes of Reinforcement Learning[1]

Yue Wu

March 23, 2026

[1] Course ORCS6529 of Columbia University

Outline

Introduction
- MDP
- Solve an MDP when MDP is known

RL: Solve an MDP when MDP is unknown
- Reinforcement Learning
- Q-learning(deterministic policy)
- TD-learning(deterministic policy)
- Policy gradient methods(stochastic policy)

Outline

Introduction
- MDP
- Solve an MDP when MDP is known

RL: Solve an MDP when MDP is unknown
- Reinforcement Learning
- Q-learning(deterministic policy)
- TD-learning(deterministic policy)
- Policy gradient methods(stochastic policy)

Markov Decision Process

$$(\mathcal{S}, \mathcal{A}, R, P)$$

At $t = 0, 1, 2, ...$, observe s_t, take action a_t, observe reward r_t and next state s_{t+1}.

- $\mathcal{S}$: State space
- $\mathcal{A}$: Action space
- R: Reward
- P: Transition prob matrix

If we have finite S states and A actions, then the dimension of Reward is $R \in \mathcal{R}^{S \times A \times S}$, the dimension of Transition prob matrix is $P \in \mathcal{R}^{S \times A \times S}$

Goal of MDP: Expectation of total Reward

For a given policy, the Expectation of total Reward is:

$$V^{\pi}(s) = \mathbb{E}\left(\sum_{t=1}^{\infty} \gamma^{t-1} r_t \,\middle|\, s_1 = s; a_t = \pi(s_t)\right)$$

which satisfies:

$$V^{\pi}(s) = \mathbb{E}_{a \sim \pi(s), s' \sim P(s,a)}\left[R(s,a,s') + \gamma V_{\gamma}^{\pi}(s')\right]$$

Markov property and Markovian policy

Policy: what action to take at any time step.
Dimension of a policy for a given start state:

Goal of MDP - find a policy to maximize reward

Find a stationary policy that maximize the expectation of the total reward of $t = 1, 2, ...$

$$V^{\pi}(s) = \mathbb{E}\left(\sum_{t=1}^{\infty} \gamma^{t-1} r_t \,\middle|\, s_1 = s; a_t = \pi(s_t) \right)$$

$$\pi^* = \arg\max_{\pi} V^{\pi}(s), \quad \forall s \in \mathcal{S}$$

Outline

Introduction
- MDP
- Solve an MDP when MDP is known

RL: Solve an MDP when MDP is unknown
- Reinforcement Learning
- Q-learning(deterministic policy)
- TD-learning(deterministic policy)
- Policy gradient methods(stochastic policy)

How to find the optimal policy/value

When we know the (R, P) of MDP, this is just a simple equation to solve. But technically, how?

- Linear programming
- Iterative algorithms
  - Value iteration: Iteratively improve estimate of optimal value(a vector of length S) $[V^*(s), s \in \mathcal{S}]$
  - Q-value iteration: Iteratively improve estimate of Q-values(a tabular of size $S \times A$) $[Q^*(s, a), s \in \mathcal{S}, a \in \mathcal{A}]$
  - Policy iteration: Iteratively improve estimate of optimal policy(a vector of length S) $[\pi^*(s_1), ..., \pi^*(S)]$)

Q-value

Q-values qualifies the first action under a certain policy for following steps. The introduction of Q-value make it possible for us to get the optimal policy by optimal value without knowing transition model P by $\pi(s) = \arg\max_a Q(s, a)$.

Q^*: the expected utility on taking action a in state s, and thereafter acting optimally

$$\begin{aligned} Q^*(s, a) &:= R(s, a) + \gamma \sum_{s' \in S} P(s, a, s') V^*(s') \\ &= R(s, a) + \gamma \sum_{s' \in S} P(s, a, s') \left(\max_{a'} Q^*(s', a') \right) \end{aligned}$$

Q-value iteration

Iteratively improve estimate of Q-values.

1. Initialize $Q^0 \in \mathbb{R}^{S \times A}$
2. In every iteration $k = 0, 1, 2, \ldots$, improve the Q-value as

$$Q^k(s, a) = R(s, a) + \gamma \sum_{s'} P(s, a, s') \left(\max_{a'} Q^{k-1}(s', a') \right) \quad \forall s, a$$

3. Stop if $\|Q^k - Q^{k-1}\|_\infty$ is small
4. Output policy π_k, where

$$\pi_k(s) = \arg\max_a Q^k(s, a)$$

Policy Iteration using Q-values

Iteratively improve estimate of optimal policy.

1. Initialize policy π^0.
2. In every iteration $k = 0, 1, 2, \ldots,$

- (Policy evaluation) Compute $Q^{\pi^k}(s, a), \ \forall s, a$
- (Greedy Policy improvement) Compute new policy

$$\pi^{k+1}(s) := \arg\max_a Q^{\pi^k}(s, a), \forall s$$

3. Stop when $\pi^{k+1} = \pi^k$ or not much change in policy.

Outline

Introduction
MDP
Solve an MDP when MDP is known

RL: Solve an MDP when MDP is unknown
Reinforcement Learning
Q-learning(deterministic policy)
TD-learning(deterministic policy)
Policy gradient methods(stochastic policy)

Reinforcement Learning

- Learning from observations

  When we do not know the (R, P) of the MDP of interest, we can interact with the environment to estimate them.

$$R(s, a) = \mathbb{E}(r_t \mid S_t = s, a_t = a)$$

$$P(s, a, s') = P(S_{t+1} = s' \mid S_t = s, a_t = a)$$

- Question
  - How to solve the best policy(implement Q-value iteration and policy iteration) using samples?
  - How to interact with the environment to generate useful samples?

Outline

Introduction
- MDP
- Solve an MDP when MDP is known

RL: Solve an MDP when MDP is unknown
- Reinforcement Learning
- Q-learning(deterministic policy)
- TD-learning(deterministic policy)
- Policy gradient methods(stochastic policy)

Tabular Q-learning

Q-learning is a **sample-based version of Q-value iteration**.

- recall the update step in Q-learning

$$Q^k(s,a) = R(s,a) + \gamma \sum_{s'} P(s,a,s') \left(\max_{a'} Q^{k-1}(s',a') \right) \quad \forall s, a$$

- A naive method
  For $k = 1, 2, ...$, observe s_k, take a random action a_k, observe reward r_k and next state s_{k+1}, **improve the one-point Q-value at** (s_k, a_k) as

$$Q^k(s_k, a_k) = r_t + \gamma \left(\max_{a'} Q^{k-1}(s_{k+1}, a') \right)$$

  - unbiased but large variance

Tabular Q-learning

Two ways to make the algorithm more steady and efficient

- Slow down the update(e.g. with learning rate α_k)

$$Q^k(s_k, a_k) = (1-\alpha_k)Q^{k-1}(s_k, a_k) + \alpha_k \left(r_k + \gamma \left(\max_{a'} Q^{k-1}(s_{k+1}, a') \right) \right)$$

- Choose action a_k smartly(e.g. ϵ-greedy)

$$a_t = \arg\max_a Q^{t-1}(s_t, a), \text{ w.p. } 1 - \epsilon$$

Large-scale Q-learning

If we are facing large state space, the Q-tabular would become huge and hard to update. How to reduce the dimension? By **parameterizing tabular Q-values into a model**

$$Q(s, a) = f_\theta(x(s, a))$$

where the vector $x(s, a)$(feature embedding) can have less dimension, and $f_\theta(\cdot)$ can be any function of the vector(e.g. Linear regression or DNN). So **updating Q tabular** $\rightarrow$ **updating** θ **by gradient of loss**.

$$Q^k(s_k, a_k) = r_t + \gamma(\max_{a'} Q^{k-1}(s_{k+1}, a'))$$

e.g. Loss is defined by

$$L(\theta_k) = (f_{\theta_k}(x(s_k, a_k)) - [r_k + \gamma(\max_{a'} f_{\theta_k}(s_{k+1}, a'))])^2$$

$$\theta_{k+1} = \theta_k - \alpha_k \nabla_{\theta|\theta_k} L(\theta_k)$$

Large-scale Q-learning

Some tips

Outline

Introduction
- MDP
- Solve an MDP when MDP is known

RL: Solve an MDP when MDP is unknown
- Reinforcement Learning
- Q-learning(deterministic policy)
- TD-learning(deterministic policy)
- Policy gradient methods(stochastic policy)

Tabular TD-learning

TD-learning is a **sample-based approximate version of policy evaluation**.

Outline

Introduction
- MDP
- Solve an MDP when MDP is known

RL: Solve an MDP when MDP is unknown
- Reinforcement Learning
- Q-learning(deterministic policy)
- TD-learning(deterministic policy)
- Policy gradient methods(stochastic policy)

Policy gradient methods

Q-learning and TD-learning:

- deterministic policy
- Q-value-based method
- Limitation: an arbitrarily small change in the estimated value can cause the action to be or not to be selected, hard to establish convergence assurances

Policy gradient methods:

- **stochastic policy**
- use **function form of policy** for directly finding optimal policy

Policy gradient methods

Parameterizing the policy space as

$$\pi_\theta : S \times A \to R, \ \sum_a \pi_\theta(s, a) = 1$$

- Examples
  - Discrete action space. Let $\theta = (\theta_1, ..., \theta_A)$ denote parameters for A actions, f can be any model

$$\pi_\theta(s, a) = \frac{e^{f_{\theta_a}(x_s)}}{\sum_{a' \in A} e^{f_{\theta_{a'}}(x_s)}}$$

  - Continuous unrestricted action space

$$\pi_\theta(s) = \mathcal{N}(f_\theta(s), \sigma^2)$$

Therefore, goal of finding optimal policy is equivalent to finding the optimal parameter that,

$$\theta^* = \arg\max_\theta V^{\pi_\theta}(s)$$

A natural question: Can we compute/estimate the gradient?

Policy gradient methods - Finite horizon

Policy gradient methods - Infinite horizon

$$V^{\pi_\theta}(s) = \lim_{T\to\infty} \mathbb{E}\left[\sum_{t=1}^{T} \gamma^{t-1} r_t \;\middle|\; s_1 = s; a_t \sim \pi_\theta(s_t)\right]$$
$$= \sum_s d^{\pi_\theta}(s) \sum_a \pi_\theta(s,a) R(s,a)$$

where $d^{\pi_\theta}(s)$ is the total discounted prob of being in s under π

$$d^{\pi_\theta}(s) = \lim_{T\to\infty} \sum_{t=1}^{T} \gamma^{t-1} P(S_t = s \mid s_1, \pi)$$

$$\nabla_\theta V^{\pi_\theta}(s_1) = \sum_s d^{\pi_\theta}(s) \sum_a Q^{\pi_\theta}(s,a) \nabla_\theta \pi_\theta(s,a)$$
$$= \frac{1}{1-\gamma} \mathbb{E}_{s\sim(1-\gamma)d^{\pi_\theta}, a\sim\pi_\theta(s)} \left[Q^{\pi_\theta}(s,a) \nabla_\theta \log(\pi_\theta(s,a))\right]$$

where $Q^{\pi_\theta}(s,a)$ can be estimated, $\nabla_\theta \log(\pi_\theta(s,a))$ can be computed.

Policy gradient Algorithm - Infinite horizon

references